

Transformer 与大语言模型原理

Transformer

是现代AI的基石。从GPT-4到LLaMA，从BERT到文心一言，所有主流大模型都基于Transformer

Transformer 与大语言模型原理

Transformer

是现代AI的基石。从GPT-4到LLaMA，从BERT到文心一言，所有主流大模型都基于Transformer

- - -

一、Transformer 诞生的背景

1.1 前Transformer时代的问题

模型 痛点 具体表现

RNN 长距离依赖弱化 相隔50步以上的词几乎无法关联

RNN 串行计算效率低 无法并行，训练速度慢

CNN 固定窗口限制 卷积核只能捕捉局部模式

CNN 堆叠层数多才能看到远距离 需要多层堆叠才能建立长距离关联

1.2 Transformer的三大创新

1. 自注意力机制：任意两个位置直接交互，一步建立长距离关联

2. 完全并行计算：不需要串行处理序列，GPU利用率高

3. 多头注意力：不同头关注不同类型的关联（语法、语义、指代等）

1.3 "Attention is All You Need" 论文的核心贡献

2017年，Google的Vaswani等人发表了这篇改变AI历史的论文。其核心贡献不仅是提出了Transformer，更重要的是：

-

证明了注意力机制足以替代循环和卷积：在此之前，注意力只是RNN / CNN的辅助组件（如Bahdanau的Seq2Seq模型）

- 确立了"缩放点积注意力"的标准形式： $Attention(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$ ，这个公式至今仍是所有大模型的基础。

- 开启了Scaling

Law时代：Transformer架构的可并行性使得训练超大规模模型成为可能，直接催生了GPT-3

- - -

二、自注意力机制详解

2.1 逐步计算示例

让我们用一个具体例子演示自注意力的计算过程：

假设序列长度 = 3，嵌入维度 = 4：

```
import numpy as np
```

输入序列（3个token，每个4维）

```
X = np.array([
    [1, 0, 1, 0], # token 1: "我"
    [0, 1, 0, 1], # token 2: "爱"
    [1, 1, 0, 0], # token 3: "AI"
])
```

可学习的投影矩阵

```
WQ = np.array([[1, 0], [0, 1], [1, 0], [0, 1]]) # (4, 2)
WK = np.array([[1, 0], [1, 1], [0, 1], [0, 0]]) # (4, 2)
WV = np.array([[0, 1], [1, 0], [1, 1], [0, 0]]) # (4, 2)
```

Step 1: 计算Q, K, V

```
Q = X @ WQ # (3, 2)
```

```
K = X @ WK # (3, 2)
```

```
V = X @ WV # (3, 2)
```

Step 2: 计算注意力分数

```
scores = Q @ K.T # (3, 3)
```

`scores[i][j]` = token i 对 token j 的关注程度

Step 3: 缩放

```
dk = K.shape[-1] # 2
```

```
scaledscores = scores / np.sqrt(dk)
```

Step 4: Softmax归一化

```
def softmax(x, axis=-1):  
    expx = np.exp(x - np.max(x, axis=axis, keepdims=True))  
    return expx / np.sum(expx, axis=axis, keepdims=True)  
  
attentionweights = softmax(scaledscores)  
attentionweights[i] = tokeni 对所有token的注意力分布
```

Step 5: 加权求和

```
output = attentionweights @ V # (3, 2)  
output[i] = tokeni 综合了所有token信息后的新表示
```

2.2 注意力权重的直觉

假设输入: "我 爱 AI 因为 它 很 强大"

↓

自注意力计算后:

- "它" 对 "AI" 的注意力权重最高 → "它" 指代 "AI"
- "强大" 对 "AI" 的注意力权重高 → "强大" 修饰 "AI"
- "爱" 对 "我" 和 "AI" 都有注意力 → "我 爱 AI"

2.3 为什么除以 $\sqrt{d_k}$?

严格的数学推导:

假设 q 和 k 的每个分量独立采样自均值为0、方差为1的分布, 则点积 $q_i \cdot k_i$ 的均值为0, 方差为 d_k (方差可加性)。

当 $d_k = 64$ 时, 点积的标准差为 $\sqrt{64} = 8$ 。如果不缩放:

- 点积值可能很大(如±24), 导致Softmax输出接近one-hot
- Softmax的梯度: $\frac{\partial \text{softmax}(z_i)}{\partial p_j} \approx 0$ 时梯度接近0

除以 $\sqrt{d_k}$ 后, 点积的方差归一化为1, Softmax输入值在合理范围内, 梯度更稳定

演示缩放的效果

```
logits_large = np.array([10, 20, 30]) # 不缩放, Softmax
```

```

problarge = softmax(logitslarge)
print(f"不缩放: {problarge}") # [~0, ~0, ~1]

logitsscaled = logitslarge / np.sqrt(64) # 缩放
probscaled = softmax(logitsscaled)
print(f"缩放后: {probscaled}") # 更均匀的分布

```

2.4 自注意力的计算复杂度分析

操作 复杂度 说明

$Q = XW_Q$ $O(n \cdot d \cdot d_k)$ 线性投影
 QK^T $O(n^2 \cdot d_k)$ 瓶颈：序列长度的平方
 Softmax $O(n^2)$ 逐行归一化
 $\text{Attn} \times V$ $O(n^2 \cdot d_v)$ 加权求和

总复杂度： $O(n^2 d + nd^2)$

- 当 $n \ll d$ 时（如BERT的 $n = 512$ ， $d = 768$ ）， nd^2 项主导
- 当 $n \gg d$ 时（如长文档处理）， $n^2 d$ 项主导，内存成为瓶颈

这就是为什么处理长上下文（128K+ tokens）需要特殊优化（如Flash Attention）

...

三、多头注意力机制

3.1 核心思想

将Q、K、V投影到多个子空间，每个子空间独立计算注意力，然后拼接结果。

为什么需要多头？

- 单头注意力只能捕捉一种关联模式
- 多头可以同时捕捉语法关联、语义关联、指代关联等
- 类似于CNN的多个卷积核提取不同特征

经验性发现：注意力头在训练后会自动分化出不同的 " 职责 "：

- 某些头关注相邻词（局部语法）

- 某些头关注远距离指代（如 "它" → "AI"）
- 某些头关注分隔符（如句号、[SEP]）
- 某些头关注稀有词（专有名词）

3.2 多头注意力的PyTorch实现

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import math

class MultiHeadAttention(nn.Module):
    """ 完整的多头注意力实现 """

    def __init__(self, dmodel=512, nheads=8, dropout=0.1):
        super().__init__()
        assert dmodel % nheads == 0, "dmodel 必须能被 nheads 整除"

        self.dmodel = dmodel
        self.nheads = nheads
        self.dk = dmodel // nheads

        # 线性投影层（合并所有头的投影，效率更高）
        self.WQ = nn.Linear(dmodel, dmodel, bias=False)
        self.WK = nn.Linear(dmodel, dmodel, bias=False)
        self.WV = nn.Linear(dmodel, dmodel, bias=False)
        self.WO = nn.Linear(dmodel, dmodel, bias=False)

        self.dropout = nn.Dropout(dropout)

    def forward(self, query, key, value, mask=None):
        """
        query/key/value: (batch, seqlen, dmodel)
        mask: (batch, 1, seqlen, seqlen) 或 (batch, nheads, seqlen, seqlen)
        """
        batchsize = query.size(0)
```

```

# 1. 线性投影并分头
Q = self.WQ(query).view(batchsize, -1, self.nheads, dk)
K = self.WK(key).view(batchsize, -1, self.nheads, dk)
V = self.WV(value).view(batchsize, -1, self.nheads, dk)
# Q/K/V: (batch, nheads, seqlen, dk)

# 2. 计算注意力分数
scores = torch.matmul(Q, K.transpose(-2, -1)) / dk
# scores: (batch, nheads, seqlen, seqlen)

# 3. 应用掩码 (如因果掩码、padding掩码)
if mask is not None:
    scores = scores.masked_fill(mask == 0, float('-inf'))

# 4. Softmax归一化
attnweights = F.softmax(scores, dim=-1)
attnweights = self.dropout(attnweights)

# 5. 加权求和
context = torch.matmul(attnweights, V)
# context: (batch, nheads, seqlen, dk)

# 6. 拼接所有头并输出投影
context = context.transpose(1, 2).contiguous().view(batchsize, -1, dk)
output = self.WO(context)

return output, attnweights # 返回注意力权重便于可视化

```

3.3 多头 vs 单头的参数量对比

单头注意力 ($d_{model}=512$, $d_k=512$):

WQ: $512 \times 512 = 262,144$

WK: $512 \times 512 = 262,144$

WV: $512 \times 512 = 262,144$

WO: $512 \times 512 = 262,144$

总计: $1,048,576$

8头注意力 ($d_{model} = 512$, $d_k = 64$) :

WQ: $512 \times 512 = 262,144$ ($512 \rightarrow 8 \times 64 = 512$)

WK: $512 \times 512 = 262,144$

WV: $512 \times 512 = 262,144$

WO: $512 \times 512 = 262,144$

总计: $1,048,576$

关键发现: 参数量完全相同!

多头的优势不在于更多参数, 而在于更丰富的表示空间。

- - -

四、Transformer 完整架构与PyTorch实现

4.1 编码器 (Encoder)

单层编码器的结构:

输入

↓

多头自注意力层 (所有位置互相可见)

↓ + 残差连接

层归一化

↓

前馈神经网络 (FFN: $d_{model} \rightarrow 4d_{model} \rightarrow d_{model}$)

↓ + 残差连接

层归一化

↓

输出

层归一化 vs 批归一化:

- 批归一化: 沿batch维度归一化, 依赖batch size
- 层归一化: 沿特征维度归一化, 不依赖batch size, 更适合序列模型

Pre-Norm vs Post-Norm:

- Post-Norm (原版Transformer): Attention → Add → Layer

- P r e - N o r m (现代主流, L L a M A / G P T等) : L a y e r N o r m → A t t e n t i o n
- P r e - N o r m训练更稳定, 不需要w a r m u p, 梯度更健康

4.2 解码器 (D e c o d e r)

单层解码器比编码器多一个掩码多头自注意力层和交叉注意力层:

输入 (已生成的 t o k e n)

↓

掩码多头自注意力 (当前 t o k e n只能看到之前的 t o k e n)

↓ + 残差连接

层归一化

↓

编码器 - 解码器交叉注意力 (Q来自解码器, K / V来自编码器)

↓ + 残差连接

层归一化

↓

前馈神经网络

↓ + 残差连接

层归一化

↓

输出

掩码机制: 在自注意力计算时, 用一个上三角矩阵屏蔽未来位置:

```
def createcausalmask(seqlen):
    """创建因果掩码 (下三角矩阵)"""
    mask = torch.tril(torch.ones(seqlen, seqlen)).unsqueeze(2)
    return mask # (1, 1, seqlen, seqlen)
```

使用示例

```
mask = createcausalmask(5)
print(mask)
tensor([[[[1, 0, 0, 0, 0],
[1, 1, 0, 0, 0],
```



```
[ 1,  1,  1,  0,  0 ],
[ 1,  1,  1,  1,  0 ],
[ 1,  1,  1,  1,  1 ] ] ] ] )
```

1表示可以关注，0表示被屏蔽

4.3 完整Transformer的PyTorch实现

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import math

class PositionalEncoding(nn.Module):
    """ 正弦位置编码 """
    def __init__(self, dmodel, maxlen=5000, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(p=dropout)

        pe = torch.zeros(maxlen, dmodel)
        position = torch.arange(0, maxlen, dtype=torch.float)
        divterm = torch.exp(torch.arange(0, dmodel, 2).float() / 10000)

        pe[:, 0::2] = torch.sin(position * divterm) # 偶数维度
        pe[:, 1::2] = torch.cos(position * divterm) # 奇数维度
        pe = pe.unsqueeze(0) # (1, maxlen, dmodel)
        self.register_buffer('pe', pe)

    def forward(self, x):
        # x: (batch, seqlen, dmodel)
        x = x + self.pe[:, :x.size(1)]
        return self.dropout(x)

class FeedForward(nn.Module):
    """ 前馈神经网络 (Position-wise FFN) """
    def __init__(self, dmodel, dff=None, dropout=0.1):
        super().__init__()
```

```

dff = dff or 4 dmodel # 默认4倍扩展
self.net = nn.Sequential(
    nn.Linear(dmodel, dff),
    nn.GELU(), # 现代模型多用GELU替代ReLU
    nn.Dropout(dropout),
    nn.Linear(dff, dmodel),
    nn.Dropout(dropout),
)

```

```

def forward(self, x):
    return self.net(x)

```

```

class EncoderLayer(nn.Module):
    """Transformer编码器层(Pre-Norm版本)"""
    def __init__(self, dmodel, nheads, dff=None, dropout=0.1):
        super().__init__()
        self.norm1 = nn.LayerNorm(dmodel)
        self.attn = MultiHeadAttention(dmodel, nheads, dropout)
        self.norm2 = nn.LayerNorm(dmodel)
        self.ff = FeedForward(dmodel, dff, dropout)

```

```

    def forward(self, x, mask=None):
        # Pre-Norm: 先归一化, 再注意力/FFN, 再残差连接
        x = x + self.attn(self.norm1(x), self.norm1(x), mask)
        x = x + self.ff(self.norm2(x))
        return x

```

```

class DecoderLayer(nn.Module):
    """Transformer解码器层"""
    def __init__(self, dmodel, nheads, dff=None, dropout=0.1):
        super().__init__()
        self.norm1 = nn.LayerNorm(dmodel)
        self.selfattn = MultiHeadAttention(dmodel, nheads, dropout)
        self.norm2 = nn.LayerNorm(dmodel)
        self.crossattn = MultiHeadAttention(dmodel, nheads, dropout)

```

```

self.norm3 = nn.LayerNorm(dmodel)
self.ff = FeedForward(dmodel, dff, dropout)

def forward(self, x, encoutput, srcmask=None, tgtmask=None):
    # 自注意力 (带因果掩码)
    x = x + self.selfattn(self.norm1(x), self.norm1(x), srcmask, tgtmask)
    # 交叉注意力 (Q来自解码器, K / V来自编码器)
    x = x + self.crossattn(self.norm2(x), encoutput, srcmask, None)
    # 前馈网络
    x = x + self.ff(self.norm3(x))
    return x

class Transformer(nn.Module):
    """完整的Transformer模型"""
    def __init__(self, srcvocabsize, tgtvocabsize, dmodel,
                 nencodelayers=6, ndecodelayers=6, dff=2048, dropout=0.1,
                 maxlen=5000, padidx=0):
        super().__init__()
        self.dmodel = dmodel
        self.padidx = padidx

        # 嵌入层
        self.srcembed = nn.Embedding(srcvocabsize, dmodel)
        self.tgtembed = nn.Embedding(tgtvocabsize, dmodel)

        # 位置编码
        self.posenc = PositionalEncoding(dmodel, maxlen)

        # 编码器和解码器
        self.encoderlayers = nn.ModuleList([
            EncoderLayer(dmodel, nheads, dff, dropout)
            for _ in range(nencodelayers)
        ])
        self.decoderlayers = nn.ModuleList([
            DecoderLayer(dmodel, nheads, dff, dropout)
            for _ in range(ndecodelayers)
        ])

```

```

for i in range(ndecoderlayers)
])

# 输出层
self.outputnorm = nn.LayerNorm(dmodel)
self.outputproj = nn.Linear(dmodel, tgtvocabsize)

# 参数初始化
self.initweights()

def initweights(self):
    for p in self.parameters():
        if p.dim() > 1:
            nn.init.xavieruniform(p)

def encode(self, src, srcmask=None):
    x = self.posenc(self.srcembed(src) * math.sqrt(self.d_model))
    for layer in self.encoderlayers:
        x = layer(x, srcmask)
    return x

def decode(self, tgt, encoutput, srcmask=None, tgtmask=None):
    x = self.posenc(self.tgtembed(tgt) * math.sqrt(self.d_model))
    for layer in self.decoderlayers:
        x = layer(x, encoutput, srcmask, tgtmask)
    return self.outputnorm(x)

def forward(self, src, tgt, srcmask=None, tgtmask=None):
    encoutput = self.encode(src, srcmask)
    decoutput = self.decode(tgt, encoutput, srcmask, tgtmask)
    logits = self.outputproj(decoutput)
    return logits

def makesrcmask(self, src):
    """ 源序列padding掩码 """
    return (src != self.padidx).unsqueeze(1).unsqueeze(2)

```

```

def make_tgt_mask(self, tgt):
    """ 目标序列的因果掩码 + padding掩码 """
    batchsize, seqlen = tgt.shape
    # 因果掩码
    causal_mask = torch.tril(torch.ones(seqlen, seqlen))
    causal_mask = causal_mask.unsqueeze(0).unsqueeze(0)
    # padding掩码
    pad_mask = (tgt != self.pad_idx).unsqueeze(1).unsqueeze(2)
    # 组合
    tgt_mask = causal_mask & pad_mask
    return tgt_mask

```

使用示例

```

model = Transformer(
    src_vocabsize=32000, tgt_vocabsize=32000,
    dmodel=512, nheads=8, nencoderlayers=6, ndecoderlayers=6,
    dff=2048, dropout=0.1
)

```

统计参数量

```

total_params = sum(p.numel() for p in model.parameters())
print(f"参数量: {total_params:,}") # 约 65M

```

4.4 Transformer的参数量计算

以标准的6层Transformer (dmodel=512, dff=2048, nheads=8,

组件 参数量 计算

嵌入层 32.8M $32000 \times 512 \times 2$

位置编码 0 固定不变

编码器 (6层) 18.9M $6 \times 4 \times 512^2 + 6 \times (512 \times 2048 + 2048 \times 512)$

解码器 (6层) 25.2M 编码器 $\times 1.33$ (多一个交叉注意力)

输出投影 16.4M 512×32000

总计 ~93M

关键洞察：嵌入层和输出投影占据了大量参数（约 50%），这也是为什么 ALBERT 提出嵌入分解来减少参数量。

- - -

五、位置编码详解

5.1 为什么需要位置编码？

自注意力是排列不变的（`permutation invariant`）：打乱输入顺序，输出只是相应地打乱，值不变。这意味着模型无法区分 "我爱你" 和 "我

位置编码为每个位置注入唯一的位置信息，使模型能够感知序列的顺序。

5.2 正弦位置编码

$$\begin{aligned} PE(pos, 2i) &= \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \\ PE(pos, 2i+1) &= \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right) \end{aligned}$$

为什么能编码相对位置？

对于任意固定偏移 k ， $PE(pos+k)$ 可以表示为 $PE(pos)$ 的线性变换。这允许模型通过学习到的权重来捕获相对位置关系。

```
import torch
import math

def sinusoidal_position_encoding(max_len, d_model):
    """生成正弦位置编码矩阵"""
    pe = torch.zeros(max_len, d_model)
    position = torch.arange(0, max_len).unsqueeze(1).float()
    div_term = torch.exp(torch.arange(0, d_model, 2).float() *
                           (-math.log(10000.0) / d_model))
    pe[:, 0::2] = torch.sin(position.div_term)
    pe[:, 1::2] = torch.cos(position.div_term)
    return pe
```

不同维度的频率不同：

低维度（2 i 小）→ 频率高 → 捕捉局部位置差异

高维度（2 i 大）→ 频率低 → 捕捉全局位置模式

```
pe = sinusoidalpositionencoding(100, 512)
print(f"位置编码形状: {pe.shape}") # (100, 512)
```

5.3 RoPE（旋转位置编码）——LLaMA等现代模型使用

核心问题：正弦位置编码是“加性”的（直接加到嵌入上），没有理论保证能很好地泛化到训练时未见过的序列长度。

RoPE的核心思想：通过旋转矩阵将位置信息编码到Q和K中，使内积自然包含相对位置信息。

$$q_m = R\{\theta, m\} W_q x_m, \quad k_n = R\{\theta, n\} W_k x_n$$

其中 $R\{\theta, m\}$ 是旋转矩阵：

$$R\{\theta, m\} = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & & \\ \sin m\theta_1 & \cos m\theta_1 & & \\ & & \ddots & \\ & & & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ & & & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix}$$

关键性质：

$$q_m^T k_n = x_m^T W_q^T R\{\theta, m-n\} W_k x_n$$

注意内积只依赖于相对位置 $m - n$ ，而非绝对位置。这比绝对位置编码更自然。

```
class RotaryPositionEncoding(nn.Module):
    """ 旋转位置编码 (RoPE) 实现 """
    def __init__(self, dmodel, maxlen=8192, base=10000):
        super().__init__()
        # 计算频率
        invfreq = 1.0 / (base * (torch.arange(0, dmodel, 2).float() + 1))
        self.register_buffer('invfreq', invfreq)

        # 预计算旋转角度
        t = torch.arange(maxlen).float()
        freqs = torch.outer(t, invfreq) # (maxlen, dmodel/2)
```

```

self.registerbuffer('coscache', freqs.cos()) # (n
self.registerbuffer('sincache', freqs.sin()) # (n

def forward(self, x, seqlen=None):
    """
    x: (batch, nheads, seqlen, dk)
    """
    seqlen = seqlen or x.size(2)
    dhalf = x.size(-1) // 2

    cos = self.coscache[:seqlen].unsqueeze(0).unsqueeze(1)
    sin = self.sincache[:seqlen].unsqueeze(0).unsqueeze(1)

    # 将x分为前后两半
    x1, x2 = x[:, :, :dhalf], x[:, :, dhalf:]

    # 旋转操作
    rotated = torch.cat([
        x1 * cos - x2 * sin,
        x1 * sin + x2 * cos
    ], dim=-1)

    return rotated

```

RoPE的优势：

- 自然编码相对位置关系
- 可以外推到更长序列（通过NTK-aware缩放或YaRN等技巧）
- 被LLaMA、Mistral、Qwen等主流模型采用

5.4 ALiBi (Attention with Linear Biases)

不使用位置编码，而是在注意力分数上添加与距离成比例的偏置：

$$\text{score}(i, j) = q_i^T k_j - m \cdot |i - j|$$

优势：训练时短序列，推理时可以外推到更长序列。

被 B L O O M、M P T 等模型使用，但在长上下文场景下 R o P E 表现更好。

- - -

六、B E R T：编码器模型

6.1 B E R T 的预训练任务

掩码语言模型（M L M）：

随机遮蔽 15% 的 t o k e n，模型预测被遮蔽的 t o k e n。

输入："我 [M A S K] 学习 [M A S K] 智"

目标：预测 [M A S K] = "在" 和 [M A S K] = "人"

15% 的选择策略：

- 80% 替换为 [M A S K]
- 10% 替换为随机词（防止 [M A S K] 从未在微调中出现）
- 10% 保持不变（提供正确的输入信号）

为什么不是 100% 替换为 [M A S K] ？

如果总是替换为 [M A S K]，模型可能学到一种捷径：只要看到 [M A S K] 就预测，而不需要真正理解上下文。随机替换和 10% 保持不变迫使模型对每个位置都进行语义理解。

下一句预测（N S P）：

判断两个句子是否在原文中相邻。

输入：[C L S] 句子A [S E P] 句子B [S E P]

标签：I s N e x t / N o t N e x t

N S P 的争议：R o B E R T a 的研究表明 N S P 对下游任务帮助不大，因为 N S P 任务过于简单（随机句子对 Order Prediction），要求判断两个连续句子的顺序是否被交换，更有挑战性。

6.2 B E R T 微调范式

任务 微调方式 添加的层 输入格式

文本分类 [C L S] 的输出 → 分类头 线性层 [C L S] 文本 [S E P]

命名实体识别 每个 token 的输出 $\rightarrow$ 标签 线性层 + CRF [CLS] 文本 [SEP]
 问答系统 预测答案的起止位置 两个线性层 [CLS] 问题 [SEP] 文章 [SEP]
 语义相似度 两个句子编码后比较 线性层 [CLS] 句子A [SEP] 句子B [SEP]

6.3 BERT变体详解

变体 改进 效果 核心理念

RoBERTa 去掉NSP、更大batch、更多数据、动态Mask 全面超过BERT "BERT被
 ALBERT 参数共享、嵌入分解 参数减少90%，效果相当 跨层参数共享 + 嵌入分解 ($E=128$)
 DistilBERT 知识蒸馏 体积减少40%，速度提升60% 用BERT做teacher训练小模型
 DeBERTa 解耦注意力、增强Mask 多项SOTA 内容-位置解耦注意力
 SpanBERT 随机遮蔽连续片段 抽取式任务更好 遮蔽整个span而非随机token

ALBERT的嵌入分解：

标准BERT：嵌入矩阵 $E \in \mathbb{R}^{V \times H}$ ，当 $V=30000$ ， $H=768$ 时，参数量约 $30000 \times 768 = 23040000$

ALBERT：将嵌入分解为两步 $E = E_1 \times E_2$ ，其中 $E_1 \in \mathbb{R}^{E \times H_1}$ ， $E_2 \in \mathbb{R}^{H_1 \times H}$ ， $E=128$ 时，参数量约 $30000 \times 128 + 128 \times 768 = 3993600$

七、GPT：解码器模型

7.1 自回归生成

GPT的核心：给定前文，预测下一个token。

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t | x_1, \dots, x_{t-1})$$

自回归的局限性：只能从左到右建模，无法利用后文信息。但GPT证明了规模足够大时，单向模型也能涌现出强大的理解能力。

7.2 GPT系列演进

版本 参数量 训练数据 核心创新 关键能力

GPT-1 1.17亿 5GB 验证预训练+微调范式 特定任务微调后表现好

GPT-2 15亿 40GB Zero-shot能力涌现 无需微调即可完成多种任务

GPT-3 1750亿 570GB In-context Learning Few-shot学习,

InstructGPT 1750亿 +人类反馈 RLHF对齐 遵循指令, 更安全

GPT-4 未公开 更大 多模态、推理能力飞跃 接近人类水平的推理

o1 未公开 — 强化学习驱动推理 深度推理、数学、编程

7.3 GPT架构的核心组件

输入 token IDs

↓

Token Embedding + Position Embedding

↓

×N个解码器层:

├ Layer Norm

├ 掩码多头自注意力 (Causal Self-Attention)

├ 残差连接

├ Layer Norm

├ 前馈神经网络 (GELU激活)

└ 残差连接

↓

Layer Norm

↓

线性投影 → Softmax → 下一个 token 的概率分布

GPT vs BERT架构对比:

维度 GPT BERT

注意力类型 单向 (因果掩码) 双向

预训练任务 下一词预测 (CLM) 掩码语言模型 (MLM)

架构 仅解码器 仅编码器

优势 生成能力强 理解能力强

代表 GPT-4、LLaMA BERT、RoBERTa

7.4 In-context Learning (上下文学习)

不需要更新参数，仅在提示中给几个示例，模型就能学会新任务：

请根据示例完成分类：

示例 1： " 这部电影太精彩了 " → 正面

示例 2： " 服务态度极差 " → 负面

示例 3： " 物超所值 " → ？

模型输出： 正面

为什么 `In-context Learning` 有效？

- 大模型在预训练时学会了 " 从上下文中学习 " 的模式
- 本质上是在执行一种 " 隐式的梯度下降 " (`Akyürek et al., 2022`)
- 预训练数据中包含大量 " 给定示例→输出结果 " 的模式
- 模型规模越大，`ICL` 能力越强（涌现能力）

`ICL` 的几种模式：

模式 说明 示例

`Zero-shot` 不给示例，直接描述任务 " 请判断这句话的情感：... "

`One-shot` 给 1 个示例 " 示例：好评→正面 \n 请判断：... "

`Few-shot` 给多个示例 给 3 - 5 个示例后判断

`Chain-of-Thought` 示例包含推理过程 " 因为...所以答案是... "

7.5 `Chain-of-Thought`（思维链）

让模型 " 一步一步思考 "，大幅提升推理能力：

普通提示：

Q： 餐厅有 23 个苹果，用了 20 个做午餐，又买了 6 个，还有几个？

A： 9

`CoT` 提示：

Q： 餐厅有 23 个苹果，用了 20 个做午餐，又买了 6 个，还有几个？

A： 原来有 23 个苹果。用了 20 个，还剩 $23 - 20 = 3$ 个。又买了 6 个，现在有 $3 + 6 = 9$ 个。答案是 9

C o T为什么有效？

- 将复杂推理分解为多个简单步骤
- 每一步的计算量更小，更容易准确
- 类似于人类 " 打草稿 " 的过程
- 涌现阈值：C o T在模型参数量超过约 6 0 B时效果显著

C o T的变体：

变体 方法 优势

A u t o - C o T 让模型自动生成推理链 无需人工标注

S e l f - C o n s i s t e n c y 采样多条推理链，取多数投票 更鲁棒

T r e e o f T h o u g h t 探索多个推理路径，回溯 更适合规划任务

L e a s t - t o - M o s t 先分解子问题再逐步解决 更可控

- - -

八、现代大模型架构详解

8 . 1 L L a M A架构

L L a M A（M e t a ， 2 0 2 3）是当前最流行的开源大模型基础架构，几乎所有开源模型都基于其改进：

L L a M A相对于原始T r a n s f o r m e r的改进：

组件 原始T r a n s f o r m e r L L a M A 改进原因

归一化 L a y e r N o r m R M S N o r m 计算更快，不需要计算均值

激活函数 R e L U S w i G L U 效果更好，门控机制

位置编码 正弦编码 R o P E 自然编码相对位置

注意力 M H A G Q A（L L a M A - 2 / 3） 减少K V C a c h e，加速推理

偏置项 有偏置 无偏置 减少参数，效果相当

R M S N o r m v s L a y e r N o r m：

```
class RMSNorm(nn.Module):
```

```
    """ RMS归一化——比LayerNorm更快 """
```

```
    def __init__(self, dmodel, eps=1e-6):
```

```

super().init()
self.weight = nn.Parameter(torch.ones(dmodel))
self.eps = eps

def forward(self, x):
# LayerNorm: (x - mean) / std gamma + beta
# RMSNorm: x / RMS(x) gamma (省去均值计算和偏置)
rms = torch.sqrt(torch.mean(x ** 2, dim=-1, keepdim=True))
return x / rms * self.weight

```

SwiGLU激活函数：

$$\text{SwiGLU}(x, W, V, b) = (xW \odot \text{SiLU}(xV)) + b$$

其中 $\text{SiLU}(x) = x \odot \sigma(x)$ 是Swish函数。

```

class SwiGLU(nn.Module):
    """SwiGLU前馈网络"""
    def __init__(self, dmodel, dff=None, dropout=0.1):
        super().__init__()
        dff = dff or int(8 * dmodel / 3) # LLaMA的FFN维度
        dff = ((dff + 63) // 64) * 64 # 对齐到64的倍数，提高GPU效率

        self.w1 = nn.Linear(dmodel, dff, bias=False) # 门控权重
        self.w2 = nn.Linear(dff, dmodel, bias=False) # 输出权重
        self.w3 = nn.Linear(dmodel, dff, bias=False) # 值路径权重
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        # SwiGLU = (xW1) ⊙ SiLU(xW3), 再经过W2
        return self.dropout(self.w2(F.silu(self.w1(x)) * self.w3(x)))

```

注意SwiGLU的FFN需要3个权重矩阵 (w_1 , w_2 , w_3)，而非标准FFN的2个，因此参数量更多。为了保持总参数量不变，LLaMA将dff设为 $\frac{8}{3}d_{\text{model}}$ 而非 $4d_{\text{model}}$ 。

8.2 LLaMA完整模型配置

配置 LLaMA - 7B LLaMA - 13B LLaMA - 30B LLaMA - 65B

dmodel	4096	5120	6656	8192
nheads	32	40	52	64
nlayers	32	40	60	80
dff	11008	13824	17920	22016
vocabsize	32000	32000	32000	32000
训练tokens	1T	1T	1.4T	1.4T

参数量估算公式：

$$\$ \$ \backslash \text { text } \{ \text { Params } \} \backslash \text { approx } 12 \backslash \cdot n\{ \text { layers } \} \backslash \cdot \text { dot }$$

8.3 其他重要架构

Mistral / Mixtral：

- Mistral - 7B：使用Sliding Window Attention（滑动窗口注意力），
- Mixtral 8×7B：MoE架构，8个专家每次选2个
- GQA（分组查询注意力）：8个KV头对应32个查询头

DeepSeek - V2 / V3：

- MLA（多头潜在注意力）：将KV Cache压缩到低维潜在空间
- DeepSeek MoE：细粒度专家 + 共享专家
- 辅助损失无关的负载均衡策略

Qwen系列：

- Qwen2：GQA + RoPE + SwiGLU + RMSNorm
- Qwen2 - MoE：MoE变体
- 支持超长上下文（128K+）

- - -

九、大模型训练关键技术

9.1 分布式训练

策略 原理 适用场景 通信量

数据并行 (DP) 每张卡有完整模型, 数据分片 模型能放入单卡 梯度同步
模型并行 (MP) 模型分片到多张卡 模型太大放不下 激活值传递
流水线并行 (PP) 按层分割到不同卡, 流水线执行 超深模型 点对点激活传递
张量并行 (TP) 单层的参数分到多张卡 单层太大 (如大 FFN) All-Reduce
ZeRO 优化器状态 + 梯度 + 参数分片 大模型 + 多卡 灵活

ZeRO (Zero Redundancy Optimizer) 三个阶段:

标准数据并行: 每张卡存储 完整模型 + 完整梯度 + 完整优化器状态

ZeRO-1: 优化器状态分片 (节省 4 倍内存)

ZeRO-2: 优化器状态 + 梯度分片 (节省 8 倍内存)

ZeRO-3: 优化器状态 + 梯度 + 参数全部分片 (节省 N 倍内存, $N = \text{GPU 数}$)

9.2 混合精度训练

FP16 / BF16 混合精度训练

前向传播用 FP16 (快)

梯度计算用 FP32 (准确)

关键: 维护一份 FP32 的 master 权重

BF16 vs FP16:

FP16: 5 位指数 + 10 位尾数 → 表示范围小, 容易溢出

BF16: 8 位指数 + 7 位尾数 → 表示范围与 FP32 相同, 精度略低

现代大模型普遍使用 BF16, 因为不需要 loss scaling

```
from torch.cuda.amp import autocast, GradScaler

scaler = GradScaler() # FP16需要; BF16不需要

for batch in dataloader:
    optimizer.zero_grad()
    with autocast(dtype=torch.bfloat16): # 或 torch.float16
        output = model(input)
        loss = criterion(output, target)

    scaler.scale(loss).backward() # 缩放 loss 防止 FP16 下溢
```



```

scaler.unscale(optimizer) # 反缩放梯度
torch.nn.utils.clip_grad_norm(model.parameters(), 1
scaler.step(optimizer) # 用FP32更新权重
scaler.update() # 更新缩放因子

```

9.3 Flash Attention

核心思想：分块计算注意力，减少GPU HBM（高带宽内存）的读写次数。

标准注意力： $QK^T \rightarrow$ 写回HBM $\rightarrow$ Softmax $\rightarrow$ 写回HBM $\rightarrow$ 乘 $V \rightarrow$ 写

Flash Attention：在SRAM中完成整个注意力计算，只需一次HBM读写

分块计算的关键——在线Softmax：

标准Softmax需要两次遍历：第一次找最大值，第二次计算。Flash Attention使用"在线Softmax"算法，逐块更新最大值和累积和，实现单遍历Softmax

效果：

- 2 - 4 倍训练加速
- 内存使用减少5 - 20倍
- 精确计算（不是近似）
- 已成为训练大模型的标准组件

使用Flash Attention (PyTorch 2.0+)

只需在注意力计算时设置 `attnimplementation="flashattention2"`

```

from transformers import AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-2-7b",
    torchdtype=torch.bfloat16,
    attnimplementation="flashattention2" # 启用Flash At
)

```

9.4 Scaling Laws

Kaplan et al. (2020) 的发现：

$L(N) \approx (N_c/N)^{\{\alpha_N\}}$

其中 N 是参数量, $N \approx 1.7 \times 10^{13}$, α

三个独立的 Scaling Law:

$L(N) \propto N^{-0.076}$, $L(D) \propto D^{\alpha}$

其中 N = 参数量, D = 数据量, C = 计算量。

Chinchilla 发现 (2022):

最优训练应该使计算量相等分配给模型大小和数据量:

- 给定计算预算 C , 最优参数量 $N \propto C^{0.5}$
- 训练 token 数 $D \propto C^{0.5}$
- 之前的大模型普遍 " 过参数化 " —— GPT-3 用 175B 参数只训练了 300B tokens, 远未达到最优

Chinchilla 最优的实际影响:

模型 参数量 训练 tokens 是否 Chinchilla 最优

GPT-3 175B 300B 否 (过参数化)

Chinchilla 70B 1.4T 是

LLaMA-7B 7B 1T 否 (过数据化, 但小模型更实用)

LLaMA-65B 65B 1.4T 接近最优

关键洞察: LLaMA 选择 " 过数据化 " 策略——用远超 Chinchilla 最优的数据量训练较小的模型。高, 但推理成本大幅降低。这在实践中更有价值, 因为推理是持续成本。

- - -

十、大模型推理优化

10.1 KV Cache

自回归生成时, 已生成的 token 的 K 和 V 不需要重复计算, 可以缓存。

无 KV Cache: 每步重新计算所有 token 的 K 和 V

复杂度: $O(t^2)$ 其中 t 是当前生成长度

有KV Cache：只计算新token的K和V，之前缓存的直接用

复杂度： $O(t)$ 每步只计算一个token

KV Cache的实现

```
class KVCache:
    def __init__(self):
        self.keycache = [] # List of (batch, nheads, seq len)
        self.valuecache = []

    def update(self, newkey, newvalue, layeridx):
        if len(self.keycache) <= layeridx:
            self.keycache.append(newkey)
            self.valuecache.append(newvalue)
        else:
            self.keycache[layeridx] = torch.cat([self.keycache[layeridx], newkey])
            self.valuecache[layeridx] = torch.cat([self.valuecache[layeridx], newvalue])
        return self.keycache[layeridx], self.valuecache[layeridx]
```

KV Cache的内存消耗：

KV Cache大小 = $2 \times \text{batch} \times \text{seq len} \times \text{n layers} \times \text{dmodel}$

例如：LLaMA-7B, batch=1, seq len=2048

= $2 \times 1 \times 2048 \times 32 \times 4096 \times 2 \text{ bytes (FP16)}$
= 1 GB

seq len=8192时：4 GB

seq len=32768时：16 GB

KV Cache是长上下文推理的主要内存瓶颈。

10.2 MQA / GQA

方法 K/V共享方式 KV Cache减少 效果损失

MHA 每个头独立K/V 基准 基准

MQA 所有头共享一组K/V 减少到 $1/nheads$ 略有损失

GQA 分组共享K / V (如8组) 减少到 n_{groups} / n_{heads} 几乎无损失

GQA的具体配置 (LLaMA - 2 70B) :

- $n_{heads} = 64$ (查询头数)
- $n_{kvheads} = 8$ (KV头数)
- 每组 $64 / 8 = 8$ 个查询头共享1组KV
- KV Cache减少到原来的 $8 / 64 = 12.5\%$

10.3 量化 (Quantization)

将模型权重从高精度 (FP16 / FP32) 转换为低精度 (INT8 / INT4), 减少内存和加速推理。

量化的基本原理:

$$x_{\text{quantized}} = \text{round} \left(\frac{x}{\Delta} \right) \times 2^b$$

其中 Δ 是量化步长, b 是目标位宽。

主要量化方法:

方法	原理	精度损失	推理加速
----	----	------	------

PTQ (训练后量化)	直接量化训练好的模型	较小	2 - 4x
-------------	------------	----	--------

GPTQ	逐层量化, 用Hessian信息补偿	小	3 - 4x
------	--------------------	---	--------

AWQ	保护重要权重 (激活感知)	小	3 - 4x
-----	---------------	---	--------

GGUF	CPU友好的量化格式	依配置	—
------	------------	-----	---

SmoothQuant	将激活的难度迁移到权重	小	2x
-------------	-------------	---	----

使用bitsandbytes进行4-bit量化加载

```
from transformers import AutoModelForCausalLM, BitsAndBytesConfig

quantizationconfig = BitsAndBytesConfig(
    load_in4bit=True,
    bnb4bitcomputedtype=torch.bfloat16,
    bnb4bitquanttype="nf4", # NormalFloat4, 专为正态分布权重设计
    bnb4bitusedoublequant=True, # 双重量化, 进一步节省内存
)
```

```
model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-2-7b",
    quantizationconfig=quantizationconfig,
    device_map="auto" # 自动分配到可用GPU
)
```

内存对比：

FP16： 14 GB

INT8： 7 GB

INT4： 3.5 GB

10.4 投机采样 (Speculative Decoding)

用小模型快速生成候选 token，大模型批量验证，正确的保留，错误的从大模型重新采样。

小模型生成 5 个 token → 大模型一次性验证 5 个 → 保留前 3 个正确的
→ 比大模型逐个生成快约 2 - 3 倍

关键点：

- 小模型和大模型必须使用相同的 tokenizer
- 大模型验证是并行的（一次前向传播验证多个 token）
- 如果小模型准确率高，加速效果显著
- 输出分布与大模型单独采样完全等价（无损加速）

- - -

十一、大模型训练三阶段

11.1 预训练

- 数据：海量无标注文本（数万亿 token）
- 目标：下一词预测
- 计算量：占全部训练资源的 99% +
- 结果：基础模型，有知识但不好用

预训练数据处理流水线：

原始数据（网页、书籍、代码等）

- 去重（MinHash、SimHash）
- 质量过滤（启发式规则 + 分类器）
- 去隐私（PII检测和替换）
- 分词（BPE / SentencePiece）
- 训练数据

关键的超参数：

参数 典型值 说明

学习率 $3e-4 \rightarrow 1.5e-5$ 预热+余弦衰减

批量大小 4M tokens 超大批量

序列长度 4096 逐步增加到128K

权重衰减 0.1 AdamW

梯度裁剪 1.0 全局范数裁剪

Dropout 0 预训练通常不用

11.2 监督微调（SFT）

- 数据：高质量的指令 - 回答对（数万到数十万条）
- 目标：学会按指令回答
- 效果：从“续写文本”变为“回答问题”

SFT数据的质量远比数量重要：

- LIMA论文表明：仅用1000条高质量数据微调，效果可接近GPT-4
- 数据多样性比数量更关键
- 数据质量包括：正确性、有用性、无害性

SFT训练示例（使用HuggingFace TRL库）

```
from trl import SFTTrainer, SFTConfig
from datasets import load_dataset
```

```
dataset = load_dataset("json", data_files="sftdata.")
```

```
training_args = SFTConfig(
```

```

output_dir = ". / s f t m o d e l " ,
num_train_epochs = 3 ,
per_device_train_batch_size = 4 ,
gradient_accumulation_steps = 8 , # 等效 batch_size = 32
learning_rate = 2 e - 5 , # 比预训练小 10 - 20 倍
max_seq_length = 2048 ,
packing = True , # 将短样本打包提高效率
bf16 = True ,
logging_steps = 10 ,
save_strategy = " epoch " ,
)

trainer = SFTTrainer (
model = base_model ,
args = training_args ,
train_dataset = dataset ,
processing_class = tokenizer ,
)

trainer.train ( )

```

11.3 RLHF（基于人类反馈的强化学习）

完整三步流程：

Step 1：监督微调（SFT）

用人工标注的指令 - 回答对微调基础模型

→ 得到 SFT 模型

Step 2：训练奖励模型（RM）

人类对多条回答排序 → 训练一个 " 评委 " 模型

→ 学会区分好回答和差回答

Step 3：强化学习优化（PPO）

用奖励模型作为环境，模型通过 PPO 优化生成策略

→ 生成奖励模型评分最高的回答

奖励模型训练：

```
import torch
import torch.nn as nn

class RewardModel(nn.Module):
    """奖励模型：输入(问题+回答)，输出标量分数"""
    def __init__(self, basemodel):
        super().__init__()
        self.basemodel = basemodel
        self.valuehead = nn.Linear(basemodel.config.hidden_dim, 1)

    def forward(self, input_ids, attention_mask):
        # 取最后一个token的隐状态
        outputs = self.basemodel(input_ids=input_ids, attention_mask=attention_mask)
        last_hidden = outputs.last_hidden_state[:, -1, :] # (batch, hidden_dim)
        reward = self.valuehead(last_hidden) # (batch, 1)
        return reward.squeeze(-1)

    def rewardmodel_loss(chosen_scores, rejected_scores):
        """Bradley-Terry偏好模型损失"""
        # P(chosen > rejected) = sigmoid(r_chosen - r_rejected)
        loss = -torch.log(torch.sigmoid(chosen_scores - rejected_scores))
        return loss.mean()
```

训练数据格式：

```
{
    "prompt": "解释量子力学",
    "chosen": "量子力学是描述微观粒子行为的物理学分支...", # 人类偏好
    "rejected": "量子力学就是很小的东西..." # 人类不偏好
}
```

PPO优化过程：

RLHF的PPO训练(伪代码)

每步：

1. 从prompt生成回答
2. 用奖励模型评分
3. 计算KL散度惩罚（防止偏离SFT模型太远）
4. 用PPO更新策略

```
for batch in promptdata_loader:
# 生成回答
response = model.generate(batch['prompt'])

# 奖励模型评分
reward = rewardmodel(batch['prompt'], response)

# KL散度惩罚（重要！防止reward hacking）
klpenalty = computekldivergence(model, sftmodel,

# 总奖励 = 奖励模型分数 - β × KL散度
totalreward = reward - klcoeff klpenalty

# PPO更新
ppoupdate(model, totalreward)
```

11.4 DPO (Direct Preference Optimization)

DPO跳过奖励模型，直接从偏好数据优化策略：

$$\mathcal{L}\{\text{DPO}\}(\theta) = -\mathbb{E} \left[\log \sigma \left(\frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

- y_w ：偏好回答 (chosen)
- y_l ：非偏好回答 (rejected)
- π_{ref} ：参考策略（通常是SFT模型）
- β ：控制偏离参考策略的程度

DPO的优势：

- 不需要训练奖励模型
- 不需要PPO的训练不稳定性

- 实现更简单
- 1 张卡即可训练 7 B 模型

使用TRL库进行DPO训练

```
from trl import DPOTrainer, DPOConfig

trainingargs = DPOConfig(
    outputdir = ". / dpomodel",
    perdevicetrainbatchsize = 4,
    learningrate = 5e - 7, # DPO需要非常小的学习率
    beta = 0.1, # DPO的温度参数
    maxlength = 1024,
    bf16 = True,
)

trainer = DPOTrainer(
    model = model, # 要训练的模型
    refmodel = refmodel, # 参考模型 (冻结)
    args = trainingargs,
    traindataset = preferredataset,
    processingclass = tokenizer,
)

trainer.train()
```

DPO vs RLHF对比：

维度 RLHF (PPO) DPO

奖励模型 需要单独训练 不需要

训练稳定性 不稳定 (PPO超参敏感) 稳定

计算成本 高 (4 个模型) 低 (2 个模型)

效果上限 更高 (理论更优) 稍低

实现难度 复杂 简单

推荐场景 大规模对齐 中小规模对齐

11.5 其他对齐方法

方法 原理 优势

R L A I F 用 A I 替代人类提供偏好反馈 成本低、可扩展

C o n s t i t u t i o n a l A I 定义 " 宪法 " 原则进行自我修正 可扩展、无人标注

K T O 只需二元反馈 (好 / 坏) , 无需配对偏好 数据更容易获取

O R P O 将 S F T 和对齐合并为一步 训练效率更高

S i m P O 去掉参考模型, 用序列长度归一化 更简单、更快

- - -

十二、混合专家架构 (M o E)

12.1 核心思想

M o E 只激活部分专家处理每个 t o k e n , 大幅增加总参数量但保持推理成本可控。

输入 t o k e n → R o u t e r → 选择 T o p - K 个专家 → 专家处理 → 加权合并输出

例如 8 个专家, 每次选 2 个:

- 总参数 = 8 × 专家参数
- 推理成本 ≈ 2 × 专家参数 (只有 2 个激活)

M o E 的优势:

- 模型容量大幅增加 (总参数多) , 但计算量增加很少
- 不同专家自动学习处理不同类型的 t o k e n 或知识领域
- 推理时可以通过调整激活专家数量来平衡效果和速度

12.2 M o E 的关键技术细节

R o u t e r 设计:

```
class TopKRouter(nn.Module):
    def __init__(self, dmodel, nexperts, topk=2):
        super().__init__()
        self.gate = nn.Linear(dmodel, nexperts, bias=False)
        self.topk = topk
```

```
def forward(self, x):
    # 计算路由分数
    logits = self.gate(x) # (batch, seq_len, n_experts)

    # 选择Top-K专家
    topk_logits, topk_indices = torch.topk(logits, self.k)
    topk_weights = F.softmax(topk_logits, dim=-1)

    return topk_weights, topk_indices
```

负载均衡损失：

如果不加约束，Router可能将所有token都路由到少数几个专家（"赢者通吃"）。辅助损失鼓励各专家均衡选择：

$$\mathcal{L}_{aux} = \alpha \cdot n_{experts} \sum_{i=1}^{n_{experts}} f_i^2$$

其中 f_i 是专家 i 处理的token比例， p_i 是Router分配给专家 i 的权重。

12.3 代表性MoE模型

模型	总参数	激活参数	专家数	Top-K	特点
Mixtral	8×7B	46.7B	12.9B	8	2 首个开源MoE大模型
DeepSeek-V2	236B	21B	160	6	细粒度专家+共享专家
DeepSeek-V3	671B	37B	256	8	无辅助损失的负载均衡
Qwen2.5 7B-A1.4B	57B	14B	64	8	密集+稀疏混合
DBRX	132B	36B	16	4	Databricks出品

DeepSeek的创新——细粒度专家：

传统MoE（如Mixtral）的每个专家是一个完整的FFN。DeepSeek将FFN拆分为更细粒度的专家。

- Mixtral 8×7B：8个大家长专家（每个是完整FFN）
- DeepSeek-V2：160个小孩子专家（每个是FFN的一部分），选6个

细粒度专家的优势：更灵活的组合，更好的专家specialization。

- - -

十三、大模型评估

13.1 评测基准

基准 测试能力 代表性模型分数

MMLU 多领域知识 (57 科目) GPT-4: 86.4%, LLaMA-3-70B: 82.0%

HumanEval 代码生成 GPT-4: 67%, DeepSeek-Coder-V2: 90%

GSM8K 数学推理 GPT-4: 92%, LLaMA-3-70B: 93%

MATH 高等数学 GPT-4: 52%, o1: 83%

Chatbot Arena 人类偏好排名 匿名对战投票, 最可信

13.2 评测的陷阱

- 数据污染: 测试集出现在训练数据中, 模型 "记住" 而非 "理解"
- 基准过拟合: 过度优化特定基准, 泛化能力不增反降
- 评估不完整: 代码 / 数学强不代表对话能力强
- Chatbot Arena 是目前最可信的评测方式: 人类盲评两个模型的回答

- - -

十四、前沿趋势

方向 说明 代表性进展

长上下文 从 4K → 128K → 1M token 的上下文窗口 Gemini 1.5 Pro: 1M tokens

多模态统一 文本 + 图像 + 语音 + 视频的统一模型 GPT-4V、Gemini

高效推理 量化、蒸馏、稀疏化 GPTQ、AWQ、SmoothQuant

Agent 化 从对话到自主规划和执行 ReAct、LangGraph、CrewAI

推理增强 o1 风格的推理能力提升 OpenAI o1 / o3、DeepSeek-R1

开放权重 LLaMA、Mistral、DeepSeek 等开源生态 LLaMA-3、Qwen2.5

状态空间模型 Mamba / S4 替代 Transformer Mamba-2、Jamba

数据工程 合成数据、数据质量优于数量 Phi 系列、LIMA

关于状态空间模型 (SSM):

Mamba 等 SSM 试图用线性复杂度替代 Transformer 的二次复杂度:

$O(n^2)$ \text{ (Transformer)} \quad vs \quad $O(n)$

但截至目前（2025年），纯SSM模型在长上下文和复杂推理上仍不如Transformer。混合架构（SSM + Transformer Attention）是当前的探索方向。

- - -

核心总结：Transformer通过自注意力机制实现了序列建模的范式革命。从原始Transformer Attention、更好的位置编码（RoPE）、更高效的训练和推理（混合精度 / 量化 / MoE）。大